

Ex. No. 09	ACCESSING APIS THROUGH HTTP
Date of Exercise	29/09/2026
Git Hub Host Link	https://github.com/riishaall/flutterexp9

Aim

To access a REST API using HTTP in Flutter, retrieve JSON data from the API, and display the retrieved data in the application UI.

REST API allows applications to communicate with web servers using HTTP methods. In this experiment, the HTTP package is used to send a GET request to the JSONPlaceholder REST API. The response is received in JSON format and decoded using `dart:convert`. The retrieved data is displayed in the Flutter UI using `ListView.builder`. Loading and error handling are also implemented.

Procedure:

1. Create a new Flutter project using `flutter create api_app`.
2. Add the `http` package to `pubspec.yaml`.
3. Run `flutter pub get`.
4. Import `dart:convert` and the `http` package.
5. Create an asynchronous function to send an HTTP GET request.
6. Check the HTTP response status code.
7. Decode the JSON response using `jsonDecode()`.
8. Display the retrieved data using `ListView.builder`.
9. Handle loading and error states.
10. Run the application and verify the API data.

Program

```
import 'dart:convert';
import 'package:flutter/material.dart';
import 'package:http/http.dart' as http;

void main() {
  runApp(const MyApp());
}
```

```
class MyApp extends StatelessWidget {
  const MyApp({super.key});

  @override
  Widget build(BuildContext context) {
    return MaterialApp(
      debugShowCheckedModeBanner: false,
      title: 'API Access Demo',
      theme: ThemeData(
        primarySwatch: Colors.blue,
      ),
      home: const ApiScreen(),
    );
  }
}

class ApiScreen extends StatefulWidget {
  const ApiScreen({super.key});

  @override
  State<ApiScreen> createState() => _ApiScreenState();
}

class _ApiScreenState extends State<ApiScreen> {
  List<dynamic> data = [];
  bool isLoading = true;
  String? errorMessage;

  @override
  void initState() {
    super.initState();
    fetchData();
  }

  Future<void> fetchData() async {
    try {
      final response = await http.get(
        Uri.parse('https://jsonplaceholder.typicode.com/posts'),
      );

      if (response.statusCode == 200) {
        setState() {
          data = jsonDecode(response.body);
          isLoading = false;
        };
      } else {
        setState() {
          errorMessage = 'Failed to load data';
          isLoading = false;
        };
      }
    }
  }
}
```

```
}  
} catch (e) {  
  setState(() {  
    errorMessage = 'Error: Unable to connect to the API';  
    isLoading = false;  
  });  
}  
}
```

```
@override  
Widget build(BuildContext context) {  
  return Scaffold(  
    appBar: AppBar(  
      title: const Text('REST API Data'),  
    ),  
    body: isLoading  
      ? const Center(  
        child: CircularProgressIndicator(),  
      )  
      : errorMessage != null  
        ? Center(  
          child: Text(  
            errorMessage!,  
            style: const TextStyle(fontSize: 18),  
          ),  
        )  
        : ListView.builder(  
          itemCount: data.length,  
          itemBuilder: (context, index) {  
            return ListTile(  
              leading: CircleAvatar(  
                child: Text('${data[index]['id']}'),  
              ),  
              title: Text(  
                data[index]['title'],  
              ),  
            );  
          },  
        ),  
  );  
}
```

24CS2016 Building Applications using Flutter

Output

URK23CS1124

```
PS D:\test> cd api_app
PS D:\test\api_app> flutter pub get
Resolving dependencies...
Downloading packages...
  cupertino_icons 1.0.9 (2.0.0 available)
+ http 1.6.0
+ http_parser 4.1.2
  material_color_utilities 0.13.0 (0.13.1 available)
  meta 1.18.3 (1.19.0 available)
  test_api 0.7.12 (0.7.14 available)
+ typed_data 1.4.0
  vector_math 2.4.0 (2.4.3 available)
+ web 1.1.1
Changed 4 dependencies!
5 packages have newer versions incompatible with dependency constraints.
Try `flutter pub outdated` for more information.
PS D:\test\api_app>
```

API Access Demo

localhost:53419

REST API Data

- 1 sunt aut facere repellat provident occaecati excepturi optio reprehenderit
- 2 qui est esse
- 3 ea molestias quasi exercitationem repellat qui ipsa sit aut
- 4 eum et est occaecati
- 5 nesciunt quas odio
- 6 doloreum magni eos aperiam quia
- 7 magnam facilis autem
- 8 doloreum dolore est ipsam
- 9 nesciunt iure omnis dolorem tempora et accusantium
- 10 optio molestias id quia eum
- 11 et ea vero quia laudantium autem
- 12 in quibusdam tempore odit est dolorem
- 13 dolorum ut in voluptas mollitia et saepe quo animi
- 14 voluptatem eligendi optio

```
pubspec.yaml X main.dart X
api_app > ! pubspec.yaml
1  name: api_app
2  description: "A new Flutter project."
3  # The following line prevents the package from being accidentally published to
4  # pub.dev using `flutter pub publish`. This is preferred for private packages.
5  publish_to: 'none' # Remove this line if you wish to publish to pub.dev
6
7  # The following defines the version and build number for your application.
8  # A version number is three numbers separated by dots, like 1.2.43
9  # followed by an optional build number separated by a +.
10 # Both the version and the builder number may be overridden in flutter
11 # build by specifying --build-name and --build-number, respectively.
12 # In Android, build-name is used as versionName while build-number used as versionCode.
13 # Read more about Android versioning at https://developer.android.com/studio/publish/versioning
14 # In iOS, build-name is used as CFBundleShortVersionString while build-number is used as CFBundleVersion.
15 # Read more about iOS versioning at
16 # https://developer.apple.com/library/archive/documentation/General/Reference/InfoPlistKeyReference/Articles/CoreFour
17 # In Windows, build-name is used as the major, minor, and patch parts
18 # of the product and file versions while build-number is used as the build suffix.
19 version: 1.0.0+1
20
21 environment:
22   sdk: ^3.13.2
23
24 # Dependencies specify other packages that your package needs in order to work.
25 # To automatically upgrade your package dependencies to the latest versions
26 # consider running `flutter pub upgrade --major-versions`. Alternatively,
27 # dependencies can be manually updated by changing the version numbers below to
28 # the latest version available on pub.dev. To see which dependencies have newer
29 # versions available, run `flutter pub outdated`.
30 dependencies:
31   flutter:
32     sdk: flutter
33
34 # The following adds the Cupertino Icons font to your application.
35 # Use with the CupertinoIcons class for iOS style icons.
36 cupertino_icons: ^1.0.8
37 http: ^1.5.0
```

Result

Thus, the REST API was successfully accessed using HTTP in Flutter. The JSON data retrieved from the REST API was decoded and displayed in the Flutter user interface using ListView.builder. Loading and error handling were also implemented successfully.